

Foreman/Katello Hands-On Lab Guide

Foreman/Katello Hands-On Lab Guide

Homelab: satellite.homelab.local (192.168.50.3) — RHEL 9 (containerized Foreman/Katello)

Since `hammer` isn't behaving in your container setup, every task below now gives you **three ways in**: UI, `hammer` CLI (with a fix-it section first), and raw **REST API via `curl`** as a guaranteed fallback that works regardless of container quirks. Use whichever is reliable for you — the API path never breaks because it just talks HTTPS to Satellite directly.

Fix Hammer First (or Skip It)

Why hammer usually breaks in a container deployment:

1. **Hammer isn't installed where you're typing the command.** If Foreman/Katello runs *inside* the container but you're typing `hammer` on the RHEL 9 host, it won't exist there unless you `dnf install rubygem-hammer_cli_katello` on the host too, or `exec` into the container:

```
podman ps                                # or: docker ps - find the foreman/katello container name
podman exec -it <container_name> hammer ping
```

2. **No hammer config / wrong host.** Hammer needs `/etc/hammer/cli.modules.d/foreman.yml` (or `~/.hammer/cli_config.yml`) pointing at the right host. Check:

```
cat /etc/hammer/cli.modules.d/foreman.yml
```

It should have `:host: 'https://satellite.homelab.local'`. If it says `localhost` but Foreman is in a container mapped to a different port/host, that's your glitch.

3. **SSL cert not trusted.** Self-signed certs cause `hammer` to hang or throw SSL errors. Quick test:

```
hammer --no-headers ping
# or force insecure just to confirm it's a cert issue (don't leave this on):
hammer -u admin -p 'yourpass' --insecure ping
```

If `--insecure` fixes it, the real fix is trusting the CA:

```
sudo cp /etc/pki/katello/certs/katello-server-ca.crt /etc/pki/ca-trust/source/anchors/
sudo update-ca-trust
```

4. **Credentials not cached.** Hammer prompts for user/pass every time unless configured. Fastest one-off test:

```
hammer -u admin -p 'yourpass' ping
```

If that works but bare `hammer ping` doesn't, it's a config/session issue, not a broken install.

If none of that resolves it — don't fight it. Every task below has a `curl`-based API equivalent that talks straight to Satellite's REST API over HTTPS. It's actually what `hammer` does under the hood, so you lose zero functionality.

API basics you'll reuse in every task:

```
# Store credentials once
export SAT_USER=admin
export SAT_PASS='yourpass'
export SAT_URL=https://satellite.homelab.local

# Generic pattern (add -k if cert isn't trusted yet)
curl -k -u $SAT_USER:$SAT_PASS -H "Accept: application/json" -H "Content-Type: application/json" \
  $SAT_URL/api/v2/<endpoint>
```

Task 1: Organization & Location Structure

UI: Administer → Organizations → New Organization → `HomelabCorp`; Administer → Locations → `DataCenter-Hyd`, `DataCenter-BLr`.

hammer:

```
hammer organization create --name "HomelabCorp" --label homelabcorp
hammer location create --name "DataCenter-Hyd"
hammer location add-organization --name "DataCenter-Hyd" --organization "HomelabCorp"
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/organizations \
  -d '{"organization": {"name": "HomelabCorp", "label": "homelabcorp"}}'

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/locations \
  -d '{"location": {"name": "DataCenter-Hyd"}}'
```

Verify: `curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/api/v2/organizations | grep name`

Real-world tie-in: Tenancy boundary — nothing else exists without an Org scope. Banks/telcos split by business unit or geography exactly this way.

Task 2: Content Management (Repos & Sync Plans)

UI: Content → Products → New Product `RockyLinux9` → New Repository (yum type) pointing at Rocky mirrors → Sync Now.

hammer:

```
hammer product create --name "RockyLinux9" --organization "HomelabCorp"
hammer repository create --name "Rocky9-BaseOS" --product "RockyLinux9" \
  --organization "HomelabCorp" --content-type yum \
  --url https://download.rockylinux.org/pub/rocky/9/BaseOS/x86_64/os/
hammer repository synchronize --name "Rocky9-BaseOS" --product "RockyLinux9" --organization "HomelabCorp"
```

curl (API fallback):

```
ORG_ID=$(curl -sk -u $SAT_USER:$SAT_PASS $SAT_URL/api/v2/organizations | python3 -c "import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/products \
  -d '{"organization_id": $ORG_ID, "name": "RockyLinux9"}'

PRODUCT_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/products?organization_id=$ORG_ID" | python3 -c "import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/repositories \
  -d '{"product_id": $PRODUCT_ID, "name": "Rocky9-BaseOS", "content_type": "yum", "url": "https://download.rockylinux.org/pub/rocky/9/BaseOS/x86_64/os/"}'

REPO_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/repositories?product_id=$PRODUCT_ID" | python3 -c "import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/repositories/$REPO_ID/sync
```

Verify: `curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/katello/api/v2/repositories/$REPO_ID | grep content_counts`

Real-world tie-in: Same reasoning as your local RPM repo experiment — mirror once, serve internally, control bandwidth and timing.

Task 3: Content Views & Lifecycle Environments

UI: Content → Lifecycle Environments: `Development` → `Testing` → `Production` (chained). Content → Content Views → New → add repos → Publish → Promote through each environment in order.

hammer:

```
hammer lifecycle-environment create --name "Development" --organization "HomelabCorp" --prior "Library"
hammer content-view create --name "Rocky9-CV" --organization "HomelabCorp"
hammer content-view add-repository --name "Rocky9-CV" --organization "HomelabCorp" \
  --product "RockyLinux9" --repository "Rocky9-BaseOS"
hammer content-view publish --name "Rocky9-CV" --organization "HomelabCorp"
hammer content-view version promote --content-view "Rocky9-CV" --organization "HomelabCorp" \
  --to-lifecycle-environment "Development" --version 1.0
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/environments \
  -d "{\"organization_id\": $ORG_ID, \"name\": \"Development\", \"prior_id\": 1}"

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/content_views \
  -d "{\"organization_id\": $ORG_ID, \"name\": \"Rocky9-CV\"}"

CV_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/content_views?organization_id=$ORG_ID" | python3 -c
"import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

# Add repo (repository_ids as array)
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X PUT \
  $SAT_URL/katello/api/v2/content_views/$CV_ID \
  -d "{\"repository_ids\": [$REPO_ID]}"

# Publish
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/content_views/$CV_ID/publish \
  -d '{"description": "Initial publish"}'
```

Note: publish/promote are asynchronous — the API returns a task ID. Poll it:

```
curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/foreman_tasks/api/tasks/<task_id>
```

Real-world tie-in: This is the core reason Satellite exists — freeze a tested package set, promote the same bits through Dev → Test → Prod, no drift.

Task 4: Activation Keys

UI: Content → Activation Keys → New → attach Lifecycle Environment `Production` + Content View `Rocky9-CV`.

hammer:

```
hammer activation-key create --name "rocky9-prod-key" \
  --lifecycle-environment "Production" --content-view "Rocky9-CV" --organization "HomelabCorp"
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/activation_keys \
  -d "{\"organization_id\": $ORG_ID, \"name\": \"rocky9-prod-key\", \"content_view_id\": $CV_ID}"
```

Real-world tie-in: One key = zero-touch registration for every new server in that lifecycle path.

Task 5: Host Registration

This step doesn't use hammer at all — it runs on the client, not Satellite, so your hammer issue doesn't affect it. It's registered via `subscription-manager` (RPM-based) or `bootstrap.py` (Ubuntu):

```
sudo rpm -Uvh https://satellite.homelab.local/pub/katello-ca-consumer-latest.noarch.rpm
sudo subscription-manager register \
  --org="HomelabCorp" --activationkey="rocky9-prod-key" \
  --serverurl=https://satellite.homelab.local:8443/rhsm
```

Ubuntu client:

```
curl -O https://satellite.homelab.local/pub/bootstrap.py
sudo python3 bootstrap.py --login=admin --organization="HomelabCorp" \
  --activationkey="ubuntu-prod-key" --server satellite.homelab.local
```

Verify (API, from Satellite):

```
curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts?organization_id=$ORG_ID"
```

Real-world tie-in: Mixed RHEL/Ubuntu fleets are the norm; hitting friction here mirrors real onboarding work.

Task 6: Host Groups

UI: Configure → Host Groups → New → attach Lifecycle Env + Content View + Activation Key.

hammer:

```
hammer hostgroup create --name "Web-Servers" --organization "HomelabCorp" \
  --lifecycle-environment "Production" --content-view "Rocky9-CV"
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/hostgroups \
  -d '{"hostgroup": {"name": "Web-Servers", "organization_ids": [$ORG_ID]}'
```

Real-world tie-in: New host provisioning = “assign to group,” everything else inherited automatically.

Task 7: Errata Management (Patch Management)

UI: Hosts → select host → Errata tab → Apply.

hammer:

```
hammer host errata list --host <hostname>
hammer host errata apply --host <hostname> --errata-ids RHSA-2026:XXXX
```

curl (API fallback):

```
HOST_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts?search=name=<hostname>" | python3 -c "import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts/$HOST_ID/errata"

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X PUT \
  $SAT_URL/api/v2/hosts/bulk/install_content \
  -d '{"organization_id": $ORG_ID, "included": {"ids": [$HOST_ID]}, "content_type": "errata", "content": ["RHSA-2026:XXXX"]}'
```

Real-world tie-in: CVE triage and selective patching — a real patch-management-team workflow, not blanket `yum update`.

Task 8: Remote Execution (Rex)

UI: Hosts → select multiple → Run Job → template Run Command - Script Default.

hammer:

```
hammer job-invocation create --job-template "Run Command - Script Default" \
  --inputs command="yum update -y" --search-query "hostgroup = Web-Servers"
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/job_invocations \
  -d '{"job_invocation": {"job_template_id": 1, "inputs": {"command": "yum update -y"}, "search_query": "hostgroup = Servers"}}'
```

(Get the correct `job_template_id` first: `curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/job_templates" | grep -i name`.)

Real-world tie-in: Fleet-wide command execution without SSH-loop scripting — Satellite’s built-in Ansible-like capability.

Task 9: Compliance (OpenSCAP)

UI-driven — no hammer needed for the scan itself, the scanning runs on the client via `oscap`:

```
sudo yum install -y openscap-scanner scap-security-guide
sudo oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis \
  --results /tmp/scan-results.xml --report /tmp/scan-report.html \
  /usr/share/xml/scap/ssg/content/ssg-rl9-ds.xml
```

Policy setup in Satellite UI: Hosts → Compliance → Policies → New Compliance Policy, assign SCAP content + host group.

curl (API fallback for policy creation):

```
curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/compliance_policies"
```

Real-world tie-in: Auditable compliance evidence for regulated industries — directly extends your STIG/hardening work.

Task 10: Provisioning Templates (PXE/Kickstart via Satellite)

UI: Infrastructure → Compute Resources → add your ESXi host; Hosts → New Host → select Compute Resource + Host Group + Kickstart template → Submit.

hammer:

```
hammer host create --name "web01" --hostgroup "Web-Servers" \
  --compute-resource "esxi-homelab" --organization "HomelabCorp" \
  --location "DataCenter-Hyd" --build true
```

curl (API fallback):

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/hosts \
  -d '{"host": {"name": "web01", "hostgroup_id": <hostgroup_id>, "compute_resource_id": <cr_id>, "organization_id": $ORG_ID, "build": true}}'
```

Real-world tie-in: Full bare-metal-to-production automation — ties Tasks 1–9 together into one flow.

Suggested order of attack

Given `hammer` is unreliable right now, do this: 1. Spend 10 minutes on the **Fix Hammer First** section above — it’s usually a 1-line config or cert fix. 2. If it’s still flaky, don’t lose more time on it — use the `curl` API path for everything. It’s the same engine `hammer` calls, so you’re not missing any real-world skill by using it instead. 3. Work Tasks 1 → 2 → 3 → 4 → 5 → 7 → 8 first (fastest path to something demoable), circle back to 6, 9, 10 after.

Tell me what error `hammer ping` actually throws (or paste it) and I’ll pinpoint the exact fix rather than guessing across all four possibilities above.